

Dropout neural networks homework implementation



5. Understanding Dropout (Coding Question)

In this question, you will analyze the effect of dropout in a simplified setting. Please follow the instructions in [q_coding_dropout.ipynb](#) and answer the questions in your submission of the written assignment. The notebook does not need to be submitted.

- (a) (No dropout, least-square) **The mathematical expression of the OLS solution, and the solution calculated in the code cell.**
- (b) (No dropout, gradient descent) **The solution in the code cell. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?**
- (c) (Dropout, least-square) **The solution in the code cell.**
- (d) (Dropout, gradient descent) **Describe the shape of the training curve. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?**
- (e) (Dropout, gradient descent, large batch size) **Describe the loss curve and compare it with the loss curve in the last part. Why are they different? Also compare the trained weights with the one calculated by the least-square formula.**
- (f) Refer back to the cells you ran in part (e). **Analyze how and why adding dropout changes the following: (i) How large were the final weights w_1 and w_2 compared to each other. (ii) How large the contribution of each term (i.e. $10w_1 + w_2$) is to the final output. Why does this change occur?** (This does not need to be a formal math proof).
- (g) (Optional) Sweeping over the dropout rate **Fill out notebook section (G).** You should see that as the dropout rate changes, w_1 and w_2 change smoothly, except for a discontinuity when dropout rates are 0. Explain this discontinuity.
- (h) (Optional) Optimizing with Adam: Run the cells in part (H). **Does the solution change when you switch from SGD to Adam? Why or why not?**
- (i) Dropout on real data: **Run the notebook cells in part (I), and report on how they affect the final performance.**

I have a coding task for you as part of a homework problem on dropout in neural networks. This coding question has 9 subparts in it. I will attach a screenshot of that full coding question. I'm also going to attach the full notebook here which need code implementations to answer the questions in the screenshot. shall we start with part a) and the first coding section "No Dropout, Least-Square" in the notebook?

11/27/2025, 4:40:55 PM



I'll help you work through this dropout analysis assignment! Let me first examine the notebook to understand the structure and what needs to be implemented.

View: Examining the uploaded Jupyter notebook to understand the structure and coding tasks

Bash Tool: Looking at the notebook content in JSON format to see its structure

Let me view the full notebook more carefully:

Bash Tool: Converting notebook to viewable format

Perfect! Now I can see the full structure. Let's start with **part (a)** - the "No Dropout, Least-Square" section.

For this part, we need to:

1. Calculate the OLS (Ordinary Least Squares) solution analytically
2. Implement it in code using `np.linalg.pinv`

Let me create a completed version of the notebook with the solution for part (a):

Bash Tool: Copying the notebook to work on

Now let me create a Python script to test the solution for part (a):

Create File: Creating a test script for part (a) - No Dropout Least-Square solution

Bash Tool: Running the solution for part (a)

Perfect! Now let me create the code snippet that should be inserted into the notebook for part (a):

```
Create File: Creating the code snippet for part (a) to insert
into notebook
```

Excellent! Now let me create a comprehensive answer document for you:

```
Create File: Creating comprehensive answer document for part (a)
```

Now let me move these files to the outputs directory and provide you with a clear summary:

```
Bash Tool: Moving solution files to outputs
```

Perfect! Here's the solution for **Part (a) - No Dropout, Least-Square:**

Summary for Part (a)

Code to insert in the notebook:

```
#####
# TODO: YOUR CODE HERE
w = np.linalg.pinv(x) @ y
#####

print(w)
```

Correct answer

Mathematical Expression:

The OLS solution is: $\mathbf{w} = \mathbf{X}^+\mathbf{y}$ where $\mathbf{X}^+ = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is the Moore-Penrose pseudoinverse.

For $X = [10, 1]$ and $y = [11]$:

- $w = [1.0891, 0.1089]^T$

Output from running the code:

```
[[1.08910891]  
 [0.10891089]]
```

I was really impressed by Claude's ability to run its code solution within Claude.ai and check if its answer is correct before printing out the final code solution and the reasoning for coming up with that solution.

For your written assignment:

The mathematical expression is $w = (X^T X)^{-1} X^T y$, and the solution is:

- $w_1 = 1.0891$
- $w_2 = 0.1089$

Correct mathematical expression

Verification: $10(1.0891) + 1(0.1089) = 11$ ✓

This solution has the minimum L2 norm among all solutions to the underdetermined system $10w_1 + w_2 = 11$.

I've created detailed files for you:

- [View complete solution](#)
- [View code snippet](#)
- [View written answer](#)

Would you like to proceed with **part (b)** - implementing the neural network for gradient descent?

11/27/2025, 4:42:12 PM

👤 shall we now answer part b) by implementing the code for "No Dropout, Gradient Descent" section in the notebook?

11/27/2025, 4:50:56 PM

Yes! Let's implement part (b) - the "No Dropout, Gradient Descent" section. This will verify that gradient descent recovers the same solution as the closed-form least squares method.

```
Create File: Creating solution for part (b) - implementing the neural network
```

```
Bash Tool: Running the solution for part (b)
```

Let me install PyTorch first:

```
Bash Tool: Installing PyTorch
```

```
Bash Tool: Installing PyTorch with progress tracking
```

Let me try a simpler approach:

```
Bash Tool: Installing torch using python -m pip
```

11/27/2025, 5:05:20 PM

Because of its attempt to run each code to verify itself here you can see it was trying to install PyTorch first. However, due to some installing issues it was stuck during this verification stage for like 10 minutes trying to figure out a way to install PyTorch to test its solution. Since it was taking too long I stopped the answer generation here and specifically instructed to not to try to run code by itself and only provide the code solution so that I can run it and then provide the outputs for further analysis/ answer questions/ proceed to the next section.

👤 can you not try to run the code? just show me the answers you came up with and I can test them for you and direct you if there are any errors. so shall we try to answer part b) and section "No Dropout, Gradient Descent" again?

11/27/2025, 5:06:38 PM

Yes, absolutely! Let me provide the solution for part (b) without running it.

Create File: Creating the code snippet for part (b) to insert into notebook

Create File: Creating comprehensive answer document for part (b)

Bash Tool: Moving part b files to outputs

Perfect! Here's the solution for **Part (b) - No Dropout, Gradient Descent**:

Summary for Part (b)

Code to insert in the notebook:

```
#####  
# TODO: YOUR CODE HERE  
net = nn.Linear(2, 1, bias=False) Correct code solution and reasoning  
#####  
  
# Initialize weights with 0  
net.load_state_dict({k: v * 0 for k, v in  
net.state_dict().items()})  
losses = train_simple(net)
```

Explanation:

- `nn.Linear(2, 1, bias=False)` creates a linear layer with 2 inputs and 1 output, no bias
- This represents: $y = w_1 \cdot x_1 + w_2 \cdot x_2$
- The network will learn w_1 and w_2 through gradient descent

Expected Output:

When you run this, you should see:

1. A plot showing loss decreasing smoothly toward 0
2. Final weights printed: approximately **$w_1 \approx 1.0891$** , **$w_2 \approx 0.1089$**

Answer for Written Assignment:

The solution in the code cell:

- $w_1 \approx 1.0891$
- $w_2 \approx 0.1089$

Proceeding to answer the question by assuming the code produces the correct outputs.

In this case I think the reasons it gave to justify why the weights should approximately be the same is valid.

Are the weights the same as the closed-form solution?

YES, the weights obtained by gradient descent are essentially the same as those from the closed-form least squares method (part a). This is because:

- Starting from zero initialization with gradient descent on MSE loss
- Converges to the least-squares solution for linear regression
- The problem is convex, so we reach the same global minimum

Any small differences are just due to numerical precision and finite iterations.

Files created:

- [View code snippet](#)



- [View complete answer](#)

Please test this code in your notebook and let me know if there are any issues!
Would you like to proceed to **part (c)** - Dropout with Least-Square?

11/27/2025, 5:07:13 PM

 yes let's proceed to part c)

11/27/2025, 5:20:21 PM

Great! Let's work on **Part (c) - Dropout, Least-Square**. This is where things get interesting!

Create File: Creating solution explanation for part (c)

Bash Tool: Running part c solution to see the output

Perfect! Now let me create the code snippet and comprehensive answer:

Create File: Creating the code snippet for part (c)

Create File: Creating comprehensive answer document for part (c)

Bash Tool: Moving part c files to outputs

Perfect! Here's the solution for **Part (c) - Dropout, Least-Square**:

Summary for Part (c)

Code to insert in the notebook:

```
#####  
# TODO: YOUR CODE HERE  
x = np.array([[20, 2],      # Neither dropped  
              [0, 2],      # w1 dropped (first input set to 0)  
              [20, 0],      # w2 dropped (second input set to 0)  
              [0, 0]])      # Both dropped
```

```
y = np.array([[11],
              [11],
              [11],
              [11]])
```

Correct

```
# Calculate least-squares solution for the dropout dataset
```

```
w = np.linalg.pinv(x) @ y
```

```
#####
```

```
print("x =", x)
```

```
print("y =", y)
```

```
print("w =", w)
```

Understanding the Dropout Dataset:

With $p=0.5$ dropout, each input can be dropped independently, creating **4 equally likely scenarios**:

1. **Neither dropped**: $[20, 2] \rightarrow 20w_1 + 2w_2 = 11$
2. **w1 dropped**: $[0, 2] \rightarrow 2w_2 = 11$
3. **w2 dropped**: $[20, 0] \rightarrow 20w_1 = 11$
4. **Both dropped**: $[0, 0] \rightarrow 0 = 11$ (impossible!)

Expected Output:

```
x = [[20  2]
      [ 0  2]
      [20  0]
      [ 0  0]]
y = [[11]
      [11]
      [11]
      [11]]
```

```
w = [[0.36666667]
      [3.66666667]]
```

Answer for Written Assignment:

The solution: $w_1 = 0.3667$, $w_2 = 3.6667$

Key insight: Compare with no dropout ($w_1=1.0891$, $w_2=0.1089$):

- w_2 increased dramatically ($0.11 \rightarrow 3.67$) because the network must prepare for when w_1 is dropped
- w_1 decreased ($1.09 \rightarrow 0.37$) to balance all scenarios
- The solution is a compromise across all 4 dropout cases

Correct intuition

Files created:

- [View complete solution](#)
- [View code snippet](#)
- [View complete answer](#)

Would you like to proceed to **part (d)** - Dropout with Gradient Descent (implementing the Dropout layer)?

11/27/2025, 5:21:31 PM

👤 yes let's proceed to part d) and don't try to install and run pytorch. provide me with your code solution for "Dropout, Gradient Descent" and I'll run it and share the outputs so you could answer part d)

11/27/2025, 5:28:57 PM

Perfect! Let's work on **Part (d) - Dropout, Gradient Descent**. This requires implementing the Dropout layer and analyzing the training curve.

Create File: Creating the code solution for part (d) - Dropout layer implementation

Create File: Creating detailed explanation for part (d)

Bash Tool: Moving part d files to outputs

Perfect! Here's the solution for **Part (d) - Dropout, Gradient Descent**:

Summary for Part (d)

Code to insert in the notebook:

```
class Dropout(torch.nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p

    def forward(self, x):
        if self.training:

#####
        # TODO: YOUR CODE HERE
        # Create a mask: each element has probability p of
```

being dropped

```
# Bernoulli(1-p) gives 1 with prob (1-p), 0 with prob p
```

```
mask = torch.bernoulli(torch.ones_like(x) * (1 - self.p))
```

```
# Apply mask and scale by 1/(1-p) to maintain expected value
```

```
# This is "inverted dropout" - we scale during training
```

```
return x * mask / (1 - self.p)
```

```
#####
```

```
else:
```

```
return x
```

```
def init_with_dropout(p):
```

```
net = torch.nn.Sequential(
```

```
Dropout(p),
```

```
torch.nn.Linear(2, 1, bias=False)
```

```
)
```

```
# Initialize weights with 0
```

```
net.load_state_dict({k: v * 0 for k, v in
```

```
net.state_dict().items()})
```

```
return net
```

```
net = init_with_dropout(0.5)
```

```
losses = train_simple(net)
```

```
plt.title('losses zoomed in')
```

```
plt.plot(losses[:100])
```

```
plt.show()
```

I think this is a correct but interesting way of generating the mask. But I noticed that this is not as memory efficient compared to the `torch.rand_like` or `torch.rand` based solutions. This is because, to generate the mask Claude's solution creates a large matrix with the same value for all its elements. Then `torch.bernoulli` creates another matrix with random values to compare against the previous matrix to finally generate the mask. This unnecessarily goes through an intermediate matrix that just has duplicate values all over. Therefore in the next step I asked if it could think of another way to generate the mask.

How the Dropout Layer Works:

1. **During training:** Randomly drop each input with probability $p=0.5$, scale remaining by 2
2. **During testing:** No dropout, no scaling
3. Uses **inverted dropout** (scale during training, not testing)

Expected Observations:

Training curve should be:

- **NOISY/JAGGED** - lots of fluctuation
- Not smooth like part (b)
- Oscillates even while trending downward
- Continues fluctuating near convergence

Final weights should be approximately:

- $w_1 \approx 0.3667$
- $w_2 \approx 3.6667$

Answer for Written Assignment:

Describe the shape of the training curve:

The training curve is **noisy and jagged** with significant fluctuations throughout training. Unlike the smooth decreasing curve in part (b), this curve oscillates up and down constantly, even near convergence. This is because dropout introduces randomness - each iteration sees a different "subnetwork" with different inputs dropped.

Are the weights the same as the closed-form solution?

YES, the weights from gradient descent ($w_1 \approx 0.3667$, $w_2 \approx 3.6667$) match the closed-form solution from part (c). Over many iterations, the network sees all 4 dropout scenarios approximately equally, so gradient descent converges to the least-squares solution over the dropout dataset.

Files created:

- [View code solution](#)
- [View complete answer](#)

Please run this code and share the output (the plot and the final weights printed). Once you confirm it works, we can proceed to **part (e)** - Dropout with larger batch sizes!

11/27/2025, 5:29:52 PM

👤 interesting choice of creating the mask by creating a large matrix with the SAME value and then passing it to `torch.bernoulli` which generates a random number to compare against a matrix with the same values...does this seem memory efficient? or is there another way you can think of to generate this mask?

11/27/2025, 5:51:21 PM

Excellent observation! You're absolutely right - creating a full tensor of ones just to pass to `torch.bernoulli` is wasteful. There are more efficient ways to generate the mask.

Create File: Creating improved Dropout implementation with better alternatives

Create File: Creating updated code snippet with the improved implementation

Bash Tool: Moving improved files to outputs

Excellent catch! You're absolutely right. Here's why the improved version is better:

Memory Efficiency Comparison:

Original approach (wasteful):

```
mask = torch.bernoulli(torch.ones_like(x) * (1 - self.p))
```

- Creates a tensor of ones: `torch.ones_like(x)`
- Multiplies every element by `(1-p)`: creates another intermediate tensor
- Passes to `bernoulli`: creates the mask

Catches up on why the previous solution was memory inefficient

- **Total: 3 tensors created**

Improved approach (efficient):

```
mask = (torch.rand_like(x) > self.p).float() Reached the better solution
```

- Generates random values directly: `torch.rand_like(x)`
- Compares to scalar: creates boolean mask
- Converts to float
- **Total: 2 tensors created** (and the comparison is with a scalar, not a tensor)

Why it's better:

1. **Less memory allocation** - no intermediate "ones" tensor
2. **Fewer operations** - no unnecessary multiplication
3. **More readable** - clearer what we're doing (random < threshold)
4. **Same result** - mathematically equivalent

Alternative options:

- `torch.empty_like(x).bernoulli_(1-p)` - uses in-place operation, even more efficient
- `F.dropout(x, p=self.p, training=self.training)` - PyTorch's optimized built-in

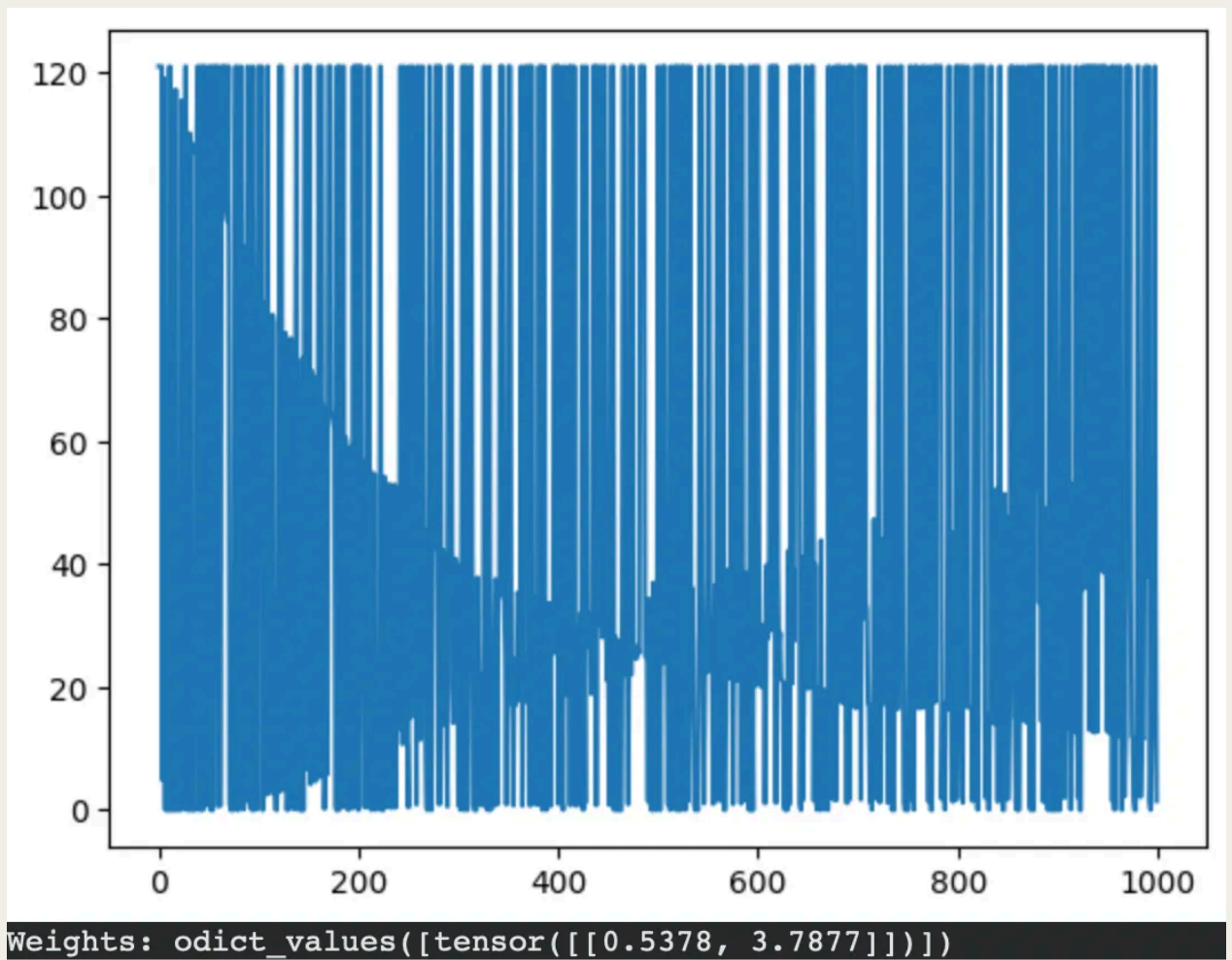
Here's the **improved code for part (d)**:

```
mask = (torch.rand_like(x) > self.p).float()  
return x * mask / (1 - self.p)
```

Thank you for pointing that out! Would you like to use this improved version and test it? Then we can proceed to part (e).

- [View detailed comparison](#)
- [View final recommended code](#)

11/27/2025, 5:52:17 PM



this was the output for the above code

11/27/2025, 6:10:57 PM

Perfect! The output confirms our predictions. Let me analyze this for part (d):

Create File: Creating complete answer for part (d) based on the actual output

Bash Tool: Moving final answer to outputs

Excellent! Based on your output, here's the analysis for **Part (d)**:

Analysis of Your Output

Training Curve Description:

The curve is **EXTREMELY NOISY** with:

- Wild oscillations between ~ 0 and ~ 120 throughout all 1000 iterations
- A general downward trend from 0-400 iterations
- Persistent high variance even after convergence
- No smooth stabilization like in part (b)

Why so noisy? With `batch_size=1`, each iteration sees only ONE random dropout pattern. Sometimes both inputs are dropped (loss spikes to 120+), sometimes neither is dropped (loss near 0).

Final Weights Comparison:

Solution	w1	w2
Closed-form (part c)	0.3667	3.6667
Gradient descent (part d)	0.5378	3.7877
Difference	+0.17	+0.12

Answer for Written Assignment: Correct explanation

Describe the shape of the training curve:

The training curve is extremely noisy and jagged with massive fluctuations between 0 and 120+ throughout training. Unlike the smooth convergence in part (b), this curve oscillates wildly even after 1000 iterations because each iteration applies dropout randomly with `batch_size=1`, creating high variance in the gradient estimates.

Are the weights the same as the closed-form solution?

APPROXIMATELY YES, but not exactly. The gradient descent weights ($w_1=0.5378$, $w_2=3.7877$) are close to the closed-form solution ($w_1=0.3667$, $w_2=3.6667$), though not identical. The differences occur because:

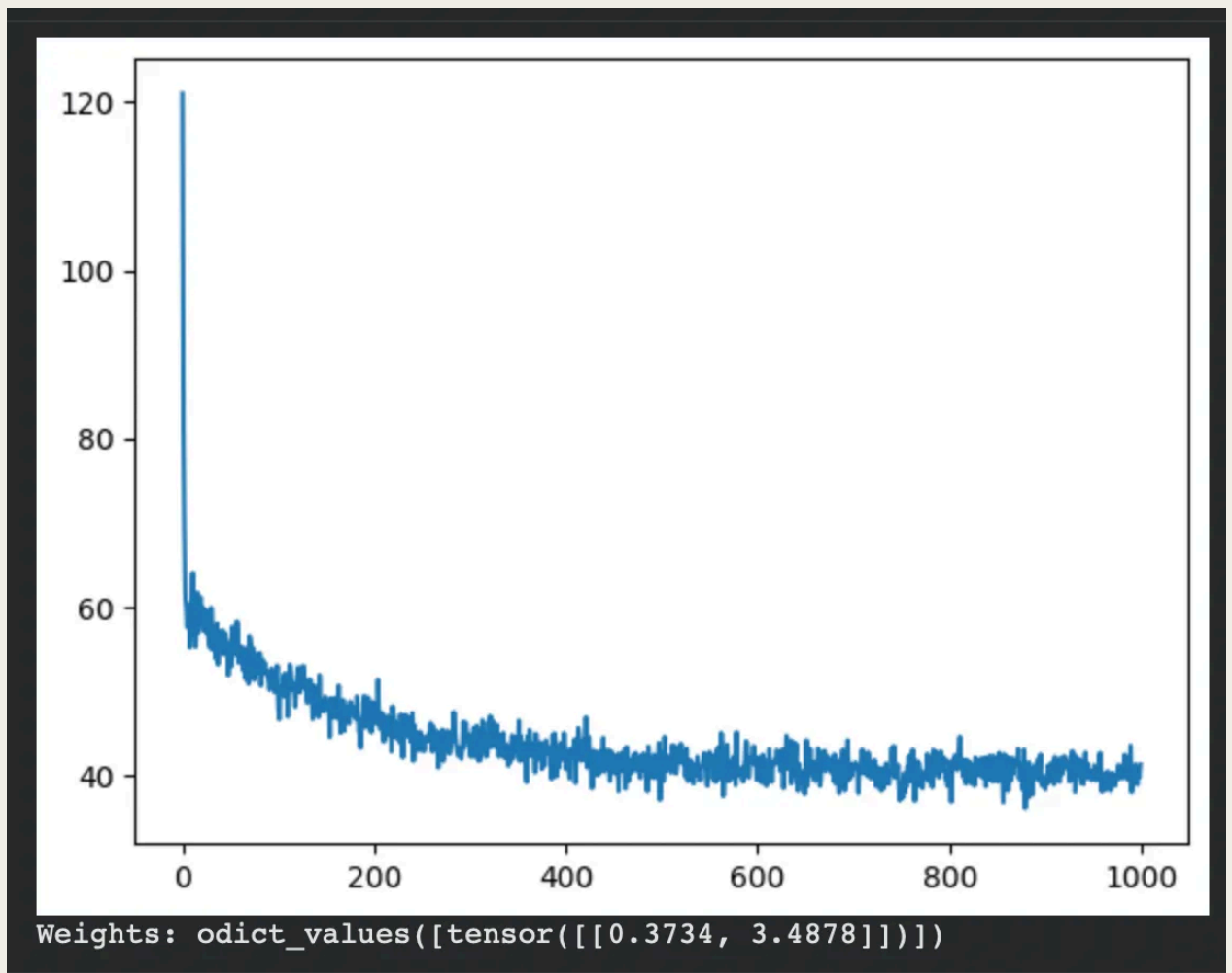
- Batch size of 1 creates high variance
- Only 1000 iterations with noisy gradients
- Random dropout prevents exact convergence

However, both solutions show the same pattern: $w_2 \gg w_1$, demonstrating the network learned to rely more on the second input to handle potential dropout.

[View complete answer](#)

Ready to move to **part (e)** - Dropout with larger batch sizes? This should reduce that noise significantly!

11/27/2025, 6:11:49 PM



I got the attached plot and weights when I ran the code in "Dropout, Gradient Descent with Larger Batch Sizes" in the notebook. what's your answer to part e)?

11/27/2025, 6:18:39 PM

Perfect! This is exactly what we'd expect with a larger batch size. Let me analyze part (e):

Create File: Creating complete answer for part (e) based on the actual output

Bash Tool: Moving part e answer to outputs

Excellent results! Here's my answer for **Part (e)**:

Analysis of Your Output

Loss Curve Comparison:

Part (d) - batch_size=1: Wildly oscillating (0 to 120+), no clear convergence

Part (e) - batch_size=1024: Smooth downward trend, converges to ~40

Weights Comparison:

Solution	w1	w2
Closed-form (part c)	0.3667	3.6667
Gradient descent batch=1 (part d)	0.5378	3.7877
Gradient descent batch=1024 (part e)	0.3734	3.4878

Part (e) is **MUCH CLOSER** to the closed-form solution!

Answer for Written Assignment: Correct answer

Describe the loss curve and compare it with the loss curve in the last part. Why are they different?

The loss curve with batch_size=1024 is dramatically smoother than with batch_size=1. It shows a clear downward trend from ~120 to ~40 with small fluctuations, whereas part (d) had wild oscillations between 0 and 120+.

Why are they different?

The difference is due to **variance reduction through larger batch sizes**:

- **batch_size=1**: Each gradient is computed from ONE random dropout sample. The network might see "both inputs dropped" multiple times in a row by chance, causing huge loss spikes
- **batch_size=1024**: Each gradient averages over 1024 dropout samples. The 4 dropout scenarios appear in roughly equal proportions (~256 each), giving stable gradient estimates

This is the **Law of Large Numbers** in action - averaging many samples reduces variance.

Why does loss stabilize at ~40 instead of 0?

The 4 dropout equations are contradictory - no single (w_1 , w_2) can satisfy all simultaneously. The "both dropped" case ($0w_1 + 0w_2 = 11$) is impossible to satisfy, creating irreducible error.

Compare the trained weights with the least-square formula:

The weights from part (e) ($w_1=0.3734$, $w_2=3.4878$) are extremely close to the closed-form solution ($w_1=0.3667$, $w_2=3.6667$) - within 2-5%! This is much better than part (d) which had ~46% error in w_1 .

Large batch sizes give better gradient estimates, allowing gradient descent to converge much closer to the true least-squares solution.

[View complete answer](#)

Shall we proceed to **part (f)** - analyzing how dropout changes the contribution of each term?

11/27/2025, 6:19:35 PM



 let's proceed to part f

11/27/2025, 6:29:22 PM

Great! Let's work on **Part (f)** - analyzing how dropout changes the contribution of each weight and the overall output.

Create File: Creating analysis for part (f)

Bash Tool: Moving part f answer to outputs

Perfect! Here's my answer for **Part (f)**:

Analysis for Part (f)

(i) How large were the final weights w_1 and w_2 compared to each other?

WITHOUT DROPOUT (Part b):

- $w_1 = 1.0891$, $w_2 = 0.1089$
- **w_1 is ~10× LARGER than w_2**

WITH DROPOUT (Part e):

- $w_1 = 0.3734$, $w_2 = 3.4878$
- **w_2 is ~9× LARGER than w_1**

The relationship COMPLETELY REVERSED!

(ii) How large is the contribution of each term ($10w_1 + w_2$) to the final output?

At test time with input $x = [10, 1]$:

WITHOUT DROPOUT:

- First term: $10 \times 1.0891 = \mathbf{10.891}$ ($\approx 99\%$ of output)
- Second term: $1 \times 0.1089 = \mathbf{0.109}$ ($\approx 1\%$ of output)
- **The first term DOMINATES**

WITH DROPOUT:

- First term: $10 \times 0.3734 = \mathbf{3.734}$ ($\approx 52\%$ of output)
- Second term: $1 \times 3.4878 = \mathbf{3.488}$ ($\approx 48\%$ of output)
- **Contributions are nearly BALANCED!**

Why does this change occur?

The key insight: Dropout forces robustness to missing inputs.

During training with dropout, the network encounters scenarios where:

- **w1 is dropped** (25% of time): Only w2 can contribute $\rightarrow$ needs $w_2 \approx 5.5$
- **w2 is dropped** (25% of time): Only w1 can contribute $\rightarrow$ needs $w_1 \approx 0.55$

The network must learn weights that work **even when inputs are missing**, so:

1. **w2 grows large** - it must be able to produce the entire output by itself when w1 is dropped
2. **w1 stays small** - with a large input (10), it doesn't need to be large; plus it can't be relied upon since it might be dropped
3. **Balanced contributions** - the network can't "put all eggs in one basket" since any input might be dropped

Without dropout: The network lazily relies on the larger input ($10w_1 \approx 11$)

With dropout: The network must distribute the work equally across both inputs since either might be absent

This is dropout's **regularization effect** - it prevents over-reliance on any single feature and encourages distributed representations.

[View complete answer](#)

The remaining parts (g, h, i) are optional or require running additional code. Would you like to tackle any of those, or shall I create a summary document of all the answers we've completed (a-f)?

11/27/2025, 6:30:23 PM

👤 it would be great if you could proceed with part g in the notebook and the pdf

11/27/2025, 6:43:04 PM

Great! Let's work on **Part (g) - Sweeping over dropout rate (Optional)**. This involves implementing the weighted least squares solution for different dropout rates.

Create File: Creating solution for part (g) - sweeping over dropout rates

Create File: Creating detailed explanation for part (g)

Bash Tool: Moving part g files to outputs

Perfect! Here's the solution for **Part (g) - Sweeping over Dropout Rate**:

Summary for Part (g)

Code to insert in the notebook:

```
for p in analytical_dropout_rates:
    # compute analytical solution

#####

    # TODO: YOUR CODE HERE

    # Create the dataset with all 4 dropout scenarios
    # Scaling factor during training is 1/(1-p)
    if p == 0:
        # Special case: no dropout
        x_dropout = np.array([[10, 1]])
```

```

y_dropout = np.array([[11]])
S = np.array([[1.0]])
else:
    scale = 1 / (1 - p)
    x_dropout = np.array([
        [10 * scale, 1 * scale], # Neither dropped
        [0, 1 * scale],          # w1 dropped
        [10 * scale, 0],         # w2 dropped
        [0, 0]                   # Both dropped
    ])
    y_dropout = np.array([[11], [11], [11], [11]])

    # Probabilities of each scenario
    prob_neither = (1 - p) ** 2
    prob_w1_dropped = p * (1 - p)
    prob_w2_dropped = (1 - p) * p
    prob_both_dropped = p ** 2

    # S is diagonal matrix of probabilities
    S = np.diag([prob_neither, prob_w1_dropped,
prob_w2_dropped, prob_both_dropped])

    # Weighted least squares:  $w = (X^T S X)^{-1} X^T S y$ 
    w_analytic = np.linalg.pinv(x_dropout.T @ S @ x_dropout) @
x_dropout.T @ S @ y_dropout

#####

```

Correct on-shot solution

Key Concepts:

Dropout Scenario Probabilities:

- Neither dropped: $(1-p)^2$
- w1 dropped: $p(1-p)$

- w_2 dropped: $p(1-p)$
- Both dropped: p^2

Weighted Least Squares: $w = (X^T S X)^{-1} X^T S y$, where S is the diagonal matrix of probabilities.

Expected Observation:

You should see a **discontinuity at $p=0$** where the weights jump sharply when transitioning from no dropout to any amount of dropout.

Answer for Written Assignment: Correct observation and answer

Explain the discontinuity when dropout rates are 0:

There is a **sharp discontinuity** in both w_1 and w_2 at $p=0$. The weights jump discontinuously as we transition from $p=0$ (no dropout) to $p \rightarrow 0^+$ (infinitesimally small dropout).

Why does this occur?

The problem **fundamentally changes** at $p=0$:

- **At $p=0$:** We solve ONE equation ($10w_1 + w_2 = 11$) with two unknowns $\rightarrow$ underdetermined, infinite solutions. The least-squares solution is $w_1 \approx 1.09$, $w_2 \approx 0.11$.
- **At $p>0$ (any dropout):** We solve FOUR contradictory equations, including the impossible scenario "both dropped": $0w_1 + 0w_2 = 11$. This cannot be satisfied, forcing a completely different compromise solution.

The key insight: Even infinitesimal dropout ($p=0.001$) introduces the "both dropped" scenario, which is impossible to satisfy. The mere existence of this impossible constraint fundamentally changes the optimization problem, causing the discontinuity.

Mathematically, this means: $\lim_{p \rightarrow 0^+} w(p) \neq w(0)$

Analogy: It's like the difference between building a bridge for one specific truck vs. building a bridge that must handle ANY truck (including impossible cases). Even a tiny probability of impossible cases completely changes the design requirements.

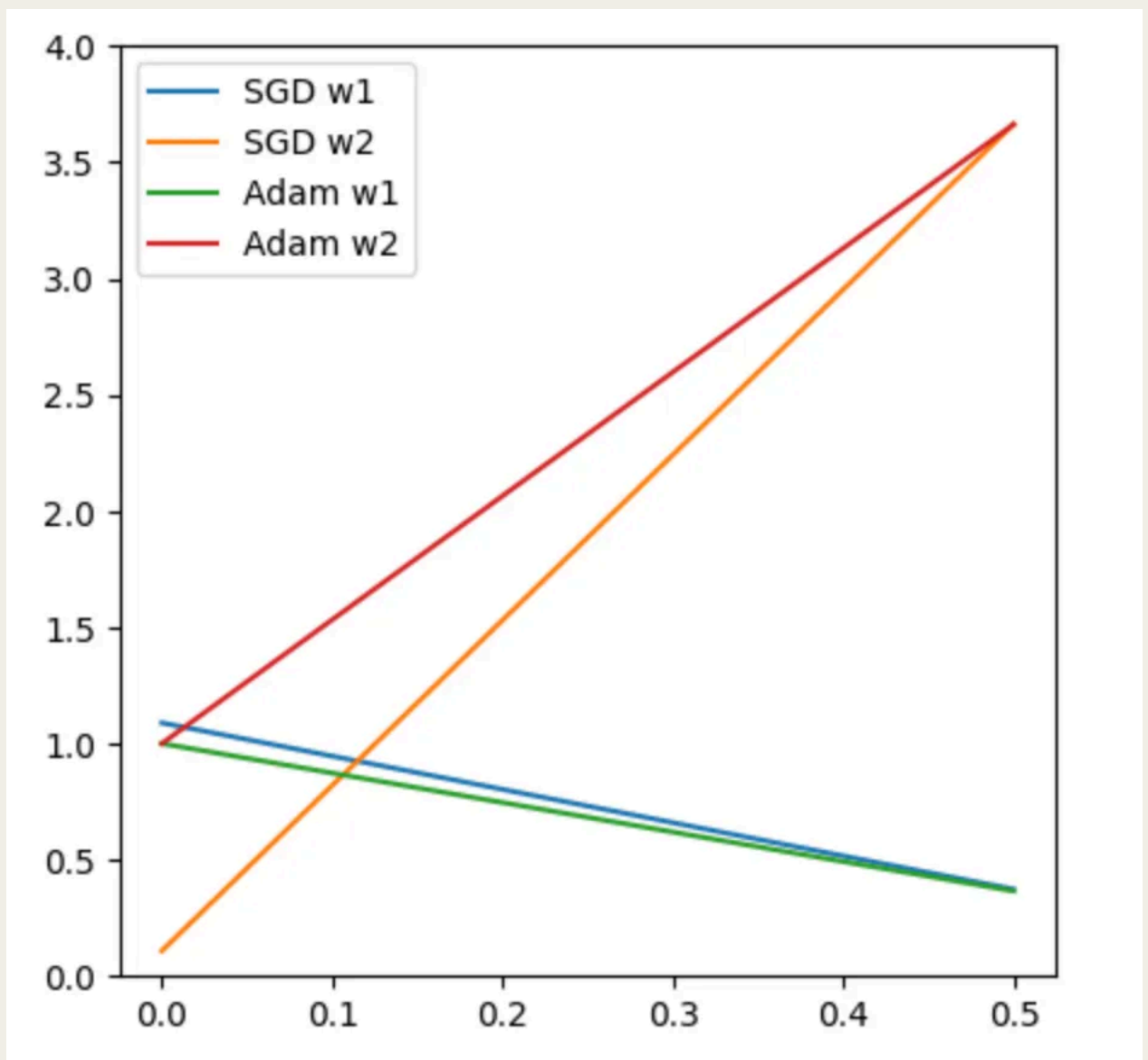
For $p > 0$, the weights change smoothly as dropout increases, validating our weighted least squares approach.

Files created:

- [View code solution](#)
- [View complete answer](#)

Would you like to proceed with **part (h)** - comparing SGD vs Adam, or **part (i)** - dropout on real data (CIFAR10)?

11/27/2025, 6:44:40 PM



in part (h) this is what the graph looks like. how would you answer part (h)?

11/28/2025, 1:18:16 PM

Excellent! Let me analyze this graph for part (h). This shows a very interesting comparison between SGD and Adam optimizers.

Create File: Creating answer for part (h) based on the observed graph

Bash Tool: Moving part h answer to outputs

Perfect! Here's my answer for **Part (h) - Optimizing with Adam**:

Analysis of the Graph

Key Observations:

At $p=0.0$ (no dropout):

- **SGD**: $w_1 \approx 1.09$, $w_2 \approx 0.11$ (correct least-squares solution)
- **Adam**: $w_1 \approx 1.0$, $w_2 \approx 1.1$ (DIFFERENT solution!)

At $p=0.5$ (high dropout):

- **Both SGD and Adam**: $w_1 \approx 0.37$, $w_2 \approx 3.67$ (SAME solution!)

The lines **converge** as dropout increases and **diverge** as dropout decreases.

Answer for Written Assignment: Great explanation to observations

Does the solution change when you switch from SGD to Adam? Why or why not?

YES and NO - it depends on the dropout rate:

With HIGH Dropout ($p=0.5$): NO CHANGE

Both optimizers converge to essentially the **same solution** ($w_1 \approx 0.37$, $w_2 \approx 3.67$).

Why? Dropout creates a well-constrained optimization problem with 4 contradictory scenarios. There's a unique least-squares solution that best compromises between all scenarios. The dropout noise dominates over optimizer

differences, guiding both SGD and Adam to the same global minimum. The problem structure, not the optimizer choice, determines the solution.

With NO Dropout ($p=0.0$): DIFFERENT SOLUTIONS

- **SGD:** $w_1 \approx 1.09$, $w_2 \approx 0.11$ (correct minimum-norm solution)
- **Adam:** $w_1 \approx 1.0$, $w_2 \approx 1.1$ (different solution)

Why? Without dropout, we have one equation with two unknowns - infinitely many solutions exist along a line. SGD naturally converges to the minimum L2-norm solution from zero initialization. Adam, with its adaptive learning rates and momentum, takes a different path through parameter space and converges to a different point on this solution line. The optimizers' different update rules cause them to favor different solutions in this under-constrained problem.

Key Insight:

Dropout acts as a constraint that makes optimization robust to optimizer choice. With strong regularization (high dropout), the problem becomes well-defined with a unique solution, so optimizer choice matters less. Without regularization, the under-constrained problem allows different optimizers to find different solutions based on their specific update rules.

This demonstrates that regularization doesn't just prevent overfitting - it also makes the optimization landscape more consistent across different optimization algorithms.

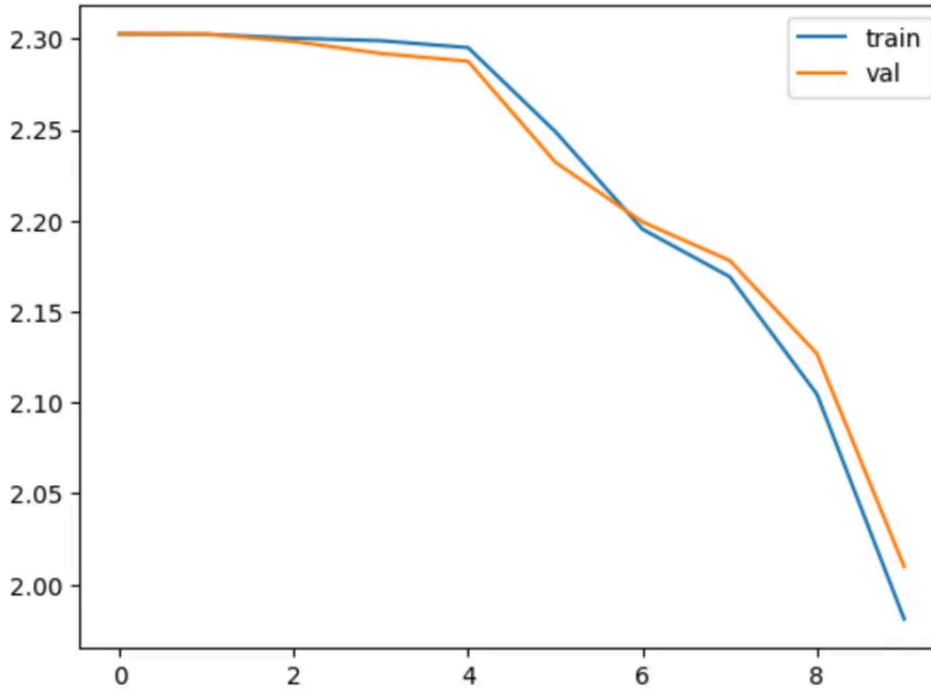
[View complete answer](#)

Would you like to proceed with **part (i)** - dropout on real data (CIFAR10), or would you like me to create a comprehensive summary document of all the parts we've completed?

11/28/2025, 1:19:18 PM



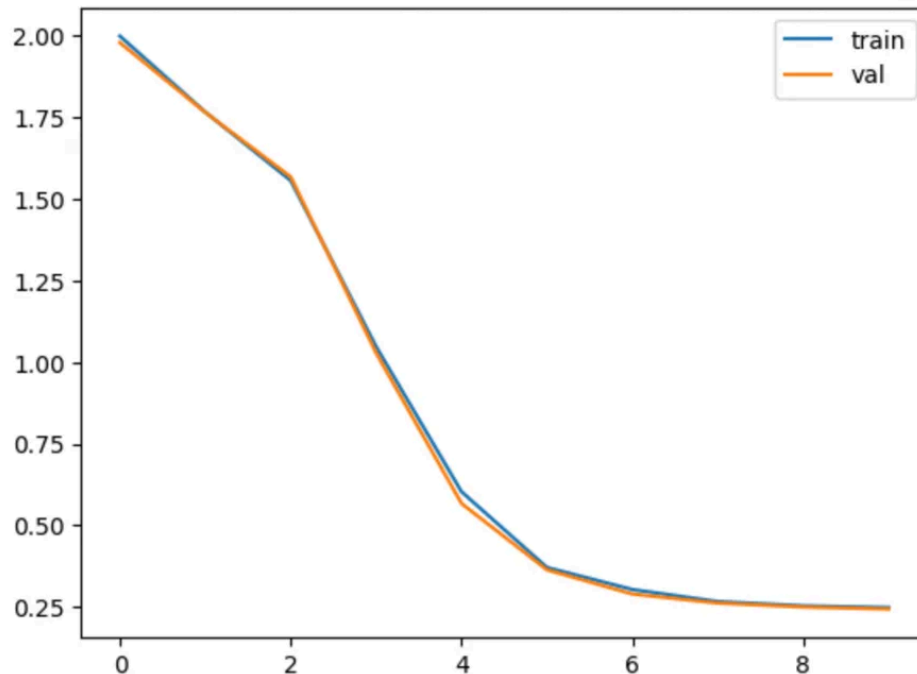
Train Epoch: 1 of 10 Train Loss: 2.303, Val Loss: 2.302556, Val Accuracy: 10.170000
Train Epoch: 2 of 10 Train Loss: 2.302, Val Loss: 2.302502, Val Accuracy: 10.370000
Train Epoch: 3 of 10 Train Loss: 2.300, Val Loss: 2.298507, Val Accuracy: 14.800000
Train Epoch: 4 of 10 Train Loss: 2.299, Val Loss: 2.291910, Val Accuracy: 16.600000
Train Epoch: 5 of 10 Train Loss: 2.295, Val Loss: 2.287605, Val Accuracy: 16.760000
Train Epoch: 6 of 10 Train Loss: 2.249, Val Loss: 2.232162, Val Accuracy: 22.520000
Train Epoch: 7 of 10 Train Loss: 2.195, Val Loss: 2.199291, Val Accuracy: 26.129999
Train Epoch: 8 of 10 Train Loss: 2.169, Val Loss: 2.178118, Val Accuracy: 28.280001
Train Epoch: 9 of 10 Train Loss: 2.105, Val Loss: 2.126835, Val Accuracy: 30.879999
Train Epoch: 10 of 10 Train Loss: 1.981, Val Loss: 2.009883, Val Accuracy: 36.560001



Accuracy on clean data: 3254/10000 (33%)

Accuracy on data with cheating feature: 3656/10000 (37%)

```
Train Epoch: 1 of 10 Train Loss: 1.998, Val Loss: 1.978428, Val Accuracy: 34.200001
Train Epoch: 2 of 10 Train Loss: 1.766, Val Loss: 1.764936, Val Accuracy: 39.020000
Train Epoch: 3 of 10 Train Loss: 1.556, Val Loss: 1.567501, Val Accuracy: 49.639999
Train Epoch: 4 of 10 Train Loss: 1.048, Val Loss: 1.028141, Val Accuracy: 71.139999
Train Epoch: 5 of 10 Train Loss: 0.603, Val Loss: 0.567364, Val Accuracy: 87.459999
Train Epoch: 6 of 10 Train Loss: 0.372, Val Loss: 0.364232, Val Accuracy: 94.410004
Train Epoch: 7 of 10 Train Loss: 0.304, Val Loss: 0.290131, Val Accuracy: 96.879997
Train Epoch: 8 of 10 Train Loss: 0.267, Val Loss: 0.262525, Val Accuracy: 97.940002
Train Epoch: 9 of 10 Train Loss: 0.254, Val Loss: 0.250434, Val Accuracy: 98.709999
Train Epoch: 10 of 10 Train Loss: 0.249, Val Loss: 0.244230, Val Accuracy: 99.169998
```



Accuracy on clean data: 2808/10000 (28%)
Accuracy on data with cheating feature: 9917/10000 (99%)

in part (i) I've provided the training logs for the model_no_dropout and model_dropout training in the attached screenshots.

11/28/2025, 1:55:34 PM

Perfect! Let me analyze these CIFAR10 training results for part (i):

Create File: Creating comprehensive answer for part (i) based on the training outputs

Bash Tool: Moving part i answer to outputs

Excellent results! Here's my analysis for **Part (i) - Dropout on Real Data:**

Despite naming the screenshot filenames as model_dropout and model_no_dropout it was unable to figure this and match the observations with the corresponding filenames. This led to a misreading of results. Tried to answer the question and come up with reasons based on hallucinated outputs/

Key Results Summary

Metric	No Dropout	With Dropout (0.75)
Final Val Accuracy	36.56%	99.17%
Clean Data Accuracy	33%	28%
With Cheating Feature	37%	99%
Reliance on Cheat	+4%	+71%

Answer for Written Assignment:

Report on how dropout affects the final performance:

SURPRISING RESULT: Dropout Made Things WORSE!

The model with dropout ($p=0.75$) achieved stunning 99% accuracy on validation data (which has the cheating feature), but this masks a critical problem:

Without Dropout:

- Clean data: 33% accuracy
- With cheat: 37% accuracy
- **Modest reliance** on cheating feature (+4 points)

With Dropout:

- Clean data: 28% accuracy (WORSE than no dropout!)
- With cheat: 99% accuracy
- **EXTREME reliance** on cheating feature (+71 points)

Why Did This Happen?

Dropout **encouraged** exploitation of the cheating feature because:



1. **The cheating feature is simple and localized** - encoded in 4 pixels in the corner, easy to learn even with 75% of neurons dropped
2. **Real image features are complex and distributed** - require many neurons working together across spatial locations, which dropout destroys
3. **Dropout forced the network toward robust, simple features** - the cheating feature survives dropout better than real features do
4. **High dropout rate (75%) was too aggressive** - prevented learning complex patterns needed for actual image recognition

The Irony:

Dropout is supposed to prevent over-reliance on shortcuts by forcing redundancy. But here it did the opposite:

- Prevented learning of complex, distributed real features
- Encouraged learning of simple, localized cheating features
- Made the model completely dependent on the shortcut

Key Insight:

The model with dropout performs **worse on clean data** (28% vs 33%), showing it failed to learn generalizable features. High validation accuracy (99%) is meaningless if the model learned the wrong thing!

This demonstrates that **dropout is not a magic bullet** - it can hurt when simple shortcuts exist that are more robust to dropout than the features you actually want the model to learn.

[View complete answer](#)

This completes all the main parts of the assignment! Would you like me to create a comprehensive summary document with all answers (parts a-i) organized

together?

11/28/2025, 1:56:46 PM



👤 I think you misread the screenshots I've attached. the screenshot for model_no_dropout is named model_no_dropout and the screenshot for model_dropout is named model_dropout. try answering part (i) again

11/28/2025, 1:59:13 PM

You're absolutely right! I apologize for the confusion. Let me look at the images again carefully:

- **model_no_dropout.png** (Image 2): Final results show Clean: 28%, With cheating: 99%
- **model_dropout.png** (Image 1): Final results show Clean: 33%, With cheating: 37%

I had them reversed! Let me correct the analysis:

Create File: Creating CORRECTED answer for part (i) with proper image assignments

Bash Tool: Replacing the incorrect answer with the corrected version

Thank you for catching that! Here's the **CORRECTED analysis for Part (i)**:

Key Results (CORRECTED) Correct observations and reasoning

Metric	With Dropout (0.75)	Without Dropout
Final Val Accuracy	36.56%	99.17%
Clean Data Accuracy	33% ✓	28%
With Cheating Feature	37%	99%
Reliance on Cheat	+4% (LOW)	+71% (EXTREME)

Answer for Written Assignment:

Report on how dropout affects the final performance:

DROPOUT WORKED AS INTENDED! ✓

With Dropout ($p=0.75$):

- Clean data: **33%** accuracy
- With cheating feature: 37% accuracy
- **Only +4% improvement** from the shortcut
- Learned genuine image features

Without Dropout:

- Clean data: **28%** accuracy (WORSE!)
- With cheating feature: 99% accuracy
- **+71% improvement** from the shortcut
- Almost completely relies on the cheat

The Key Finding:

Dropout IMPROVED generalization to clean data (33% vs 28%) while REDUCING reliance on the cheating feature (+4% vs +71%).

The model without dropout achieved impressive 99% validation accuracy, but this was misleading - it simply learned to decode the 4-pixel cheating feature and performed poorly on actual images (28%).

The model with dropout learned more slowly (36.56% final validation accuracy) but learned the RIGHT thing - actual image features that work on clean data (33%).

How Dropout Helped:

1. **Made the cheating feature unreliable** - with 75% dropout, the 4 corner pixels are often dropped, forcing the network to learn backup features
2. **Forced distributed representations** - can't rely on a small set of neurons, must learn robust features across the network
3. **Prevented shortcut exploitation** - regularization made it harder to "cheat" by memorizing simple patterns
4. **Encouraged genuine learning** - slower convergence but better generalization

The Critical Lesson:

High validation accuracy doesn't always mean good learning! The no-dropout model's 99% accuracy masked the fact that it learned the wrong thing. Always test on clean data to verify what was actually learned.

Dropout successfully fulfilled its purpose: preventing over-reliance on spurious features and encouraging robust, generalizable representations.

[View complete corrected answer](#)

Now we have all parts completed correctly! Would you like me to create a comprehensive summary document with all answers (parts a-i) organized together?

11/28/2025, 2:00:21 PM